GOTPM-CPL

(PSC3)

Technical Reference Manual

Hybrid PM + Tree Cosmological
N-Body Simulation Code with
CPL Dark Energy Support

Juhan Kim

2026

Based on the GOTPM algorithm (Dubinski et al. 2004)

https://github.com/kjhan0606/GOTPM-CPL

Contents

I Getting Started

1 Introduction 9
1.1 Key Features 9
1.2 Algorithm Overview 9

2 Quick Start Guide 11
2.1 Prerequisites 11
2.2 Building the Code 11
2.3 Setting Up a Run 11
2.3.1 Directory Structure 11
2.3.2 Parameter File 12
2.3.3 Input Power Spectrum 12
2.3.4 Job Submission (SLURM) 12

II Code Architecture

3 Parameter Reference 15
3.1 Cosmological Parameters 15
3.2 Simulation Control 15
3.3 Grid and Particle Setup 16
3.4 Power Spectrum and Normalization 16
3.5 I/O Parameters 16

4 Compile-Time Flags 17
4.1 Required Flags 17
4.2 Optional Flags 17

5 Data Structures 19
5.1 Particle Types 19
5.1.1 PM Particle (`pmparticletype`) 19
5.1.2 Tree Particle (`treeparticletype`) 19
5.1.3 Position Reconstruction Macros 20
5.2 Simulation Parameters (`SimParameters`) 20
5.3 Memory Management 20

6 Source File Reference 23
6.1 Core Simulation Loop 23

6.2 Initial Conditions 23
6.3 Force Calculation 24
6.4 Domain Decomposition and Migration 24
6.5 Cosmology and Analysis 24
6.6 I/O and Utilities 25
6.7 Header Files 25

III Algorithms

7 Initial Conditions 29
7.1 Zel'dovich Approximation (1st-Order LPT) 29
7.2 2nd-Order LPT 29

8 PM Force Calculation 31
8.1 TSC Mass Assignment 31
8.2 FFT-Based Poisson Solver 31
8.3 4th-Order FDA 31
8.4 Variable PM (VarPM) 32

9 Tree Force Correction 33
9.1 Cell-Linked-List Construction 33
9.2 Force Evaluation 33
9.3 Type Conversion 33

10 Time Integration 35
10.1 Leap-Frog Scheme 35
10.2 Half-Step Synchronization 35

11 Cosmology: CPL Dark Energy 37
11.1 CPL Parameterization 37
11.2 Expansion History 37
11.3 Growth Factor 37
11.4 Poisson Correction 37

12 Domain Decomposition and Migration 39
12.1 Slab Decomposition 39
12.2 Particle Migration 39
12.3 Load Balancing 39

IV Reference

13 Troubleshooting 43
13.1 Common Issues 43
13.1.1 Segfault in Tree Correction 43
13.1.2 Malloc Error (Extremely Large Allocation) 43
13.1.3 Power Spectrum File Not Found 43
13.2 Performance Tuning 43

14 Example: CPL Dark Energy Simulation 45
14.1 Parameter File 45
14.2 Compilation 46
14.3 Execution 46
14.4 Expected Output 46

V Post-Processing

15 FoF Halo Analysis 49
15.1 Most Bound Particle (MBP) 49
15.2 Attached Quantities 49
15.3 Center of Mass 50
15.4 Halo Shape 50

16 MBP Merger Tree 51
16.1 Basic Concepts 51
16.1.1 GID and Major GID 51
16.1.2 Merger Events 51
16.1.3 Central and Satellite MBP 51
16.1.4 MBP Mass 51
16.2 Data Structures 52
16.2.1 Original Merger Tree Format 52
16.2.2 GID-Sorted Merger Tree Format 52
16.3 Building the Merger Tree 53
16.4 Sorting by Major GID 53
16.5 Reading the Merger Tree 54
16.6 Known Issues 55
16.6.1 Disappearance of MBP 55
16.6.2 Fluctuating Small Halos 55
16.6.3 Null Values 55
16.6.4 Mass Conservation 55

17 PSB Subhalo Finder 57
17.1 Algorithm Overview 57
17.2 Density Estimation 57

17.3 Peak Detection **58**
17.4 Density Contour Separation **58**
17.5 Energy Unbinding **59**
17.6 FoF Substructure Linking **59**
17.7 MPI Parallelization **60**
17.8 Data Structures **61**
17.9 Parameters **62**
17.9.1 Compile-Time Parameters (`header.h`) 62
17.9.2 Density Grid Parameters (`sub_find_dimension.inc`) 62
17.9.3 Memory Configuration (`Rules.make`) 62
17.10 Input and Output Formats **63**
17.10.1 Input Files 63
17.10.2 Output Files 63
17.11 Building and Running **63**
17.12 Source File Reference **64**

Getting Started

1 Introduction

GOTPM-CPL (Grid-of-Oct-Tree Particle Mesh), also referred to as PSC3 (Parallel Simulation Code 3), is a massively parallel cosmological N-body simulation code that computes the gravitational evolution of dark matter particles in an expanding universe. The code implements the hybrid TreePM algorithm: long-range forces are computed on a uniform mesh using FFT-based Poisson solvers (PM), while short-range corrections are applied through a Barnes–Hut-type tree walk.

1.1 Key Features

- **Hybrid TreePM force calculation** — PM for long-range + Tree for short-range correction
- **CPL dark energy models** — Full support for the Chevallier–Polarski–Linder parameterization $w(a) = w_0 + w_a(1-a)$, including Poisson correction factors
- **2LPT initial conditions** — Built-in 2nd-order Lagrangian perturbation theory for accurate IC generation
- **MPI + OpenMP parallelization** — Slab domain decomposition with hybrid parallelism
- **Variable PM grid (VarPM)** — Adaptive z-domain boundaries for load balancing
- **Index-based position encoding** — High-precision particle positions via displacement + Lagrangian index
- **Custom memory allocator** — Stack-based pool allocator for deterministic memory management
- **Friends-of-Friends (FoF)** — Built-in parallel group finder
- **Power spectrum measurement** — On-the-fly $P(k)$ computation with TSC deconvolution

1.2 Algorithm Overview

The simulation proceeds through the following steps at each time step:

1. **PM force** — Assign particle mass to grid via TSC, solve Poisson equation in Fourier space, interpolate forces back to particles (4th-order FDA).
2. **Tree correction** — Build cell-linked-list, compute short-range force corrections using spline-softened gravity with opening angle θ.
3. **Velocity kick** — Update velocities using leap-frog integration.
4. **Position drift** — Advance particle positions.
5. **Migration** — Exchange particles between MPI domains if they have crossed slab boundaries.
6. **Domain rebalancing** — Adjust slab boundaries if load imbalance exceeds threshold.

Important

The code uses a *split-step* (leap-frog) integrator where velocities and positions are offset by half a time step. The `halfstep` structure controls the synchronization of positions and velocities for output.

2 Quick Start Guide

This chapter provides a step-by-step guide to building and running the code.

2.1 Prerequisites

- **Intel C/Fortran Compilers** — `mpiicx` and `mpiifx` (or `mpiicc/mpiifort`)
- **FFTW3** — Single-precision with MPI and OpenMP thread support (`libfftw3f`, `libfftw3f_mpi`, `libfftw3f_omp`)
- **MPI** — Intel MPI or OpenMPI
- **OpenMP** — Enabled via `-qopenmp`

2.2 Building the Code

Build Commands

```
cd PSC3_CPL_mv_new
# Edit Rules.make for your system (compiler, FFTW path, NMEG)
make clean
make this
# Produces: namu.exe
```

The build system consists of two files:

- `Rules.make` — System-specific compiler settings, library paths, and compile flags.
- `Makefile` — Object file list and build targets.

> The primary target compiler is Intel `mpiicx/mpiifx`. For PGI or GCC compilers, use the corresponding `Rules.make.*` variant (e.g., `Rules.make.pgcc`).

2.3 Setting Up a Run

2.3.1 Directory Structure

Create a separate run directory:

Run Directory Setup

```
mkdir MV_0m0.31_-1.2_-2.4
cd MV_0m0.31_-1.2_-2.4
ln -s ../PSC3_CPL_mv_new/namu.exe .
```

```
cp ../PSC3_CPL_mv_new/params.dat .
# Edit params.dat for your cosmological model
```

2.3.2 Parameter File

Edit `params.dat` to specify cosmological parameters, grid resolution, and run settings. See Chapter 3 for the full parameter reference.

2.3.3 Input Power Spectrum

Prepare an ASCII power spectrum file (e.g., from CAMB) with two columns: k [h/Mpc] and $P(k)$ [$(\mathrm{Mpc}/h)^3$]. Set `Powerflag = 2` and provide the filename in `Ascii Powerfile`.

2.3.4 Job Submission (SLURM)

Example SLURM Job Script

```
#!/bin/bash
#SBATCH --job-name=MV311224
#SBATCH --ntasks=16
#SBATCH --cpus-per-task=4
#SBATCH --time=1000000:00
#SBATCH --output outfile.o%j.grammar
#SBATCH --error errfile.e%j.grammar
#SBATCH -p normal

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun ./namu.exe params.dat
```

Choosing MPI Ranks and Threads

The number of MPI ranks (`ntasks`) determines the z-slab decomposition. The FFTW3 domain decomposition requires that N_z is divisible by the number of MPI ranks. Each rank spawns `cpus-per-task` OpenMP threads for shared-memory parallelism within the tree correction and TSC assignment.

II

Code Architecture

3 Parameter Reference

The simulation is controlled by the ASCII parameter file `params.dat`, which is read by rank 0 and broadcast to all MPI processes.

3.1 Cosmological Parameters

Parameter	Default	Description
`OmegaMatter0`	—	Present-day matter density Ω_m
`OmegaBaryon0`	—	Present-day baryon density Ω_b
`OmegaLambda0`	—	Present-day dark energy density Ω_Λ
`Hubble`	—	Hubble parameter $h = H_0/(100\ \mathrm{km/s/Mpc})$
`nPS`	—	Scalar spectral index n_s
`w0 of DE (CPL)`	-1	CPL equation of state w_0
`wa of DE (CPL)`	0	CPL evolution parameter w_a
`Sound Speed of DE`	0	Dark energy sound speed ($c_s^2 = 0$: clustering, 1: smooth)
`fNL`	0	Local non-Gaussianity amplitude
`gNL`	0	Higher-order non-Gaussianity

3.2 Simulation Control

Parameter	Default	Description
`INITIAL CONDITION`	—	1 = Zeldovich approximation, 2 = 2LPT
`Amax`	—	Maximum scale factor $a_{\max}$ (typically 48 for $z_i = 47$)
`Anow`	1	Starting scale factor
`Astep`	—	Scale factor increment per time step Δa
`Nstep`	—	Total number of time steps
`Stepcount`	1	Starting step count (for restart)
`Stepnum`	1	Step number to begin integration
`Theta`	0.3	Tree opening angle (overridden adaptively)
`Iseed`	—	Random seed for IC generation

3.3 Grid and Particle Setup

Parameter	Default	Description
`Nx, Ny, Nz`	—	PM grid dimensions (must be equal: $N_x = N_y = N_z$)
`Nspace`	1	Grid spacing in index units
`Boxsize(Mpc/h)`	—	Comoving box size in Mpc/h

3.4 Power Spectrum and Normalization

Parameter	Default	Description
`Powerflag`	—	0: generate on run, 1: CAMB binary, 2: ASCII file
`Ascii Powerfile`	—	Filename of input power spectrum
`ampRescaling flag`	0	0: none, 1: rescale to σ_8 at $z = 0$, 2: at z_{CMB}
`bias8`	1	Inverse of σ_8 for $R = 8$ Mpc$/h$

3.5 I/O Parameters

Parameter	Default	Description
`rvfilename`	—	Input RV data filename (for restart)
`rvprefix`	—	Output file prefix

4 Compile-Time Flags

The code behavior is controlled by preprocessor flags set in `Rules.make`.

4.1 Required Flags

Flag	Description
`-DINTEL`	Select Intel compiler conventions (main function name). Required for `mpiicx`.
`-DINDEX`	Enable particle index tracking. Each particle stores its Lagrangian grid index for high-precision position reconstruction.
`-DXYZDBL`	Use double-precision position reconstruction via $X = \mathrm{fmod}(x_{\mathrm{disp}} + x_{\mathrm{index}}, N_x)$. Must be paired with `-DINDEX`.
`-DINCLUDE_TREE_FORCE`	Enable tree force corrections on top of PM forces.

4.2 Optional Flags

Flag	Description
`-DNMEG=`*N*`L`	Memory pool size per MPI rank in megabytes (default: 100). Example: `-DNMEG=8000L` allocates 8 GB per rank.
`-DWGROUPSIZE=`*N*	I/O write group size for parallel output (default: 80).
`-DVarPM`	Enable variable-width PM domain decomposition. Allows non-uniform z-slab widths for load balancing.
`-DSAVESLICE`	Save x-z density slices at each time step.
`-DPMSEEDFORCE`	Use PM seed force for initial force evaluation.
`-DMPI_SLOW`	Use slower but more reliable MPI communication patterns.
`-DDEMODEL=0`	Constant Poisson correction factor (default).
`-DDEMODEL=1`	Scale-dependent Poisson correction (for k-dependent DE).
`-DMULTIMASS`	Support for multi-mass particles (adds `mass` field).
`-DINDTIME`	Individual time-stepping support (adds `tflag` field).
`-D_LARGE_FILES`	Enable large file support for I/O.

Example 4.1 (Standard Compilation)

A typical compilation command (via `Rules.make`):

```
CDFLAGS = -DWGROUPSIZE=80 -DNMEG=8000L -DINCLUDE_TREE_FORCE
    \
          -D_LARGE_FILES -DSAVESLICE -DPMSEEDFORCE
COMFLAGS = -DINDEX -DVarPM -DXYZDBL -DMPI_SLOW
OPT = -DINTEL -O2 -qopenmp
```

5 Data Structures

5.1 Particle Types

The code uses two particle structures that share the same memory through pointer casting. This is the core of the hybrid PM+Tree design.

5.1.1 PM Particle (`pmparticletype`)

Definition 5.1 (pmparticletype — 32 bytes)

```
typedef struct pmparticletype {
    float x, y, z;          /* displacement from index
        position */
    float vx, vy, vz;       /* peculiar velocity */
    long long indx;         /* Lagrangian grid index */
} pmparticletype;
```

The PM particle stores only displacement and velocity. The true position is reconstructed via:

$$X = \mathrm{fmod}\Big(x_{\mathrm{disp}} + (\mathrm{indx} \bmod m_x) + N_x,\ N_x\Big)$$

where $m_x = N_x/N_{\mathrm{space}}$. This encoding preserves sub-grid precision even for $N \geq 1024^3$.

5.1.2 Tree Particle (`treeparticletype`)

Definition 5.2 (treeparticletype — 40 bytes)

```
typedef struct treeparticletype {
    float x, y, z;
    float vx, vy, vz;
    long long indx;
    struct treeparticletype *next;  /* linked-list pointer
        */
} treeparticletype;
```

Extends `pmparticletype` with a `next` pointer for the cell-linked-list tree structure. The conversion between the two types is done in-place via backward copy (`pm2treetype`) and forward copy (`tree2pmtype`).

Important

The in-place conversion between `pmparticletype` and `treeparticletype` requires careful handling because both pointers alias the same memory. The code uses `memmove` through `char*` pointers to avoid strict-aliasing violations that can cause data corruption at `-O2` optimization.

5.1.3 Position Reconstruction Macros

```
/* With -DXYZDBL -DINDEX: */
#define XofP(p) fmod((double)((p)->x) + (double)(((p)->indx %
    simpar.mx)) + simpar.nx, simpar.nx)
#define YofP(p) fmod((double)((p)->y) + (double)((((p)->indx %
     simpar.mxmy) / simpar.mx)) + simpar.ny, simpar.ny)
#define ZofP(p) fmod((double)((p)->z) + (double)(((p)->indx /
    simpar.mxmy)) + simpar.nz, simpar.nz)
```

5.2 Simulation Parameters (`SimParameters`)

Definition 5.3 (SimParameters struct)

The global `simpar` structure holds all runtime parameters:

- **MPI identity**: `myid`, `nid`
- **Cosmology**: `omep` (Ω_m), `omeplam` (Ω_Λ), `wlam0` (w_0), `wlam1` (w_a), `CsDE` (sound speed)
- **Grid**: `nx`, `ny`, `nz`, `mx`, `my`, `mz`, `mxmy`
- **Scale factors**: `amax`, `anow`, `astep`
- **Force**: `theta` (opening angle), `pcorr` (Poisson correction)
- **Domain**: `local_nz`, `local_z_start`, `zmin`, `zmax`

5.3 Memory Management

Definition 5.4 (Stack-Based Memory Pool)

The code allocates a single large memory block at startup (`NMEG` megabytes) and manages sub-allocations through a stack-based allocator defined in `Memory2.c`:

- `Malloc(size, PPTR(var))` — Push allocation onto stack
- `Free(ptr)` — Pop or compact stack
- `Realloc(ptr, size)` — Resize with automatic compaction of subsequent allocations
- `freespace()` — Query available memory

The allocator tracks each allocation's size, starting address, and a pointer-to-pointer back-reference, enabling automatic pointer updates when memory is compacted.

The header `Memory.h` defines:

```
#define Realloc(A,B) Realloc(A,B,PPTR(A))
```

> This macro automatically supplies the third argument (pointer-to-pointer), so user code simply calls `Realloc(ptr, newsize)`.

6 Source File Reference

6.1 Core Simulation Loop

File	Description
`main.c`	Entry point. Initializes MPI/FFTW, reads `params.dat`, dispatches to `Zeldovich()` or `SecondOrderLPT()`.
`namu.zeldovich.c`	Main simulation driver for Zeldovich IC. Defines global variables (`DEFINE_SIM_PARA`, `DEFINE_PM_COMMON`, etc.), time integration loop, and type conversion functions (`pm2treetype`, `tree2pmtype`).
`namu.2lpt.c`	Main simulation driver for 2LPT IC. Similar structure to `namu.zeldovich.c` with 2nd-order displacement fields.
`treemain.c`	Tree correction driver. Calls `pm2treetype`, `treecorrection`, `PreFoF`, then `tree2pmtype`.
`pmmain.c`	PM force driver. TSC density assignment, forward FFT, Poisson solve, backward FFT, 4th-order FDA force interpolation.
`dumpmain.c`	Output controller. Triggers snapshot writes when positions and velocities are synchronized.

6.2 Initial Conditions

File	Description
`TwoLPT.F`	2nd-order LPT potential computation in Fourier space. Computes the $\phi^{(2)}$ kernel terms.
`TwoOrderLPT.F`	Converts LPT potentials to particle displacements and velocities.
`addTwoOrderLPT.c`	Applies 2nd-order displacement to particle positions.
`ZeldovichInit.F`	1st-order (Zeldovich) displacement and velocity fields.
`addZeldovichInit.c`	Applies Zeldovich displacement to particles.
`FindSeed.F`	Parallel random number seed initialization.
`sub.lynx.camb.F`	CAMB power spectrum reader and interpolator. Handles fNL non-Gaussianity.

6.3 Force Calculation

File	Description
`tsc.c` / `tsc_omp2.c`	Triangular Shaped Cloud (TSC) mass assignment with OpenMP.
`fft.c`	FFTW3-MPI wrapper for forward/backward transforms. Thread-safe initialization.
`pm_main_sub_fftw3.F`	Low-level FFTW3 workspace management routines.
`p_solver.F`	Poisson solver in Fourier space. Applies $\tilde{\phi}(k) = -\tilde{\rho}(k)/k^2$.
`fda4.c`	4th-order Finite Difference Approximation for force interpolation from grid to particles.
`VarPM.c`	Variable-resolution PM: TSC and FDA with non-uniform z-slab widths.
`Treewalk.c`	Barnes–Hut tree construction and traversal. Builds linked-list octree with opening angle criterion.
`kjhtpm.mod1.gpu.omp.c`	Tree force correction with OpenMP. Cell-linked-list construction (`ParticleCellLinkedList`) and force evaluation (`treecorrection`).
`force_spline.c`	Spline-softened gravitational force kernel.
`pcorr.c`	Poisson correction factor for CPL dark energy models.

6.4 Domain Decomposition and Migration

File	Description
`pmigrate.kisti.omp.c`	MPI particle migration between z-slabs. Handles periodic boundaries and bulk transfer.
`mydomaindecomposition.omp.c`	Adaptive load balancing. Adjusts z-slab boundaries when load imbalance exceeds 1%.
`slicemigrate.c`	FFT data migration between non-uniform and uniform z-slab layouts.
`determine_mpi_misc_param.c`	Computes FFTW3 domain layout parameters (`local_nz`, `local_z_start`).

6.5 Cosmology and Analysis

File	Description
`FLRW.F`	Friedmann equation solver. Growth factors for ΛCDM, quintessence, and CPL models. Functions: `growth()`, `growthcpl()`, `cplOmegaMatter()`, `HofEz()`, `PoissonCor()`.

File	Description
NR.F	Numerical Recipes: Simpson integration (qsimp), Romberg integration (qromo), ODE solver (odeint).
correl.F	On-the-fly power spectrum measurement with TSC window deconvolution.
PreFoF.mod2.omp.c	Parallel Friends-of-Friends group finder preparation (ghost exchange).

6.6 I/O and Utilities

File	Description
kjhrw.c	Parallel binary I/O for particle snapshots (read/write).
header.c	ASCII parameter file reader/writer.
flagwriting.c	Output scheduling: determines when to trigger data writes based on scale factor, step count, and flag files.
Memory2.c	Stack-based memory pool allocator.
timerutil.c	Wall-clock timer utilities.
indexing.c	Particle-to-Lagrangian-grid index mapping.

6.7 Header Files

File	Description
pmheader.h	Particle structures (pmparticletype, treeparticletype), SimParameters, position macros (XofP, YofP, ZofP), timer macros.
pm_common.h	Global PM variables: np, pmparticles, treeparticles, den, work.
flow.h	Evolution factors (EvolFact), half-step structure, output flag function declarations.
Memory.h	Memory allocator API and the Realloc macro.
varpm.h	Variable PM function declarations and NZBound structure.
common_simparam.h	Legacy common-block-style simulation parameters.
fortran.h	C–Fortran interface macros and FFTW/MPI includes.
kjhtree.h	Tree structure definitions.
force_spline.h	Spline kernel function declarations.

III

Algorithms

7 Initial Conditions

7.1 Zel'dovich Approximation (1st-Order LPT)

The Zel'dovich approximation displaces particles from a uniform grid according to the linear displacement field:

$$\mathbf{x}(\mathbf{q},a) = \mathbf{q} - \frac{D_1(a)}{a}\nabla\phi^{(1)}(\mathbf{q}) \tag{7.1}$$

where $\mathbf{q}$ is the Lagrangian coordinate (stored as `indx`), $D_1(a)$ is the linear growth factor, and $\phi^{(1)}$ is the gravitational potential from the linear density field.

The velocity is:

$$\mathbf{v} = \dot{a}\mathbf{x} - a\dot{D}_1\nabla\phi^{(1)} \tag{7.2}$$

7.2 2nd-Order LPT

The 2LPT adds a second-order correction:

$$\mathbf{x}(\mathbf{q},a) = \mathbf{q} - D_1(a)\Psi^{(1)}(\mathbf{q}) + D_2(a)\Psi^{(2)}(\mathbf{q}) \tag{7.3}$$

where $D_2 \approx -\frac{3}{7}D_1^2$ for $\Omega_m = 1$, and

$$\nabla^2\phi^{(2)} = \sum_{i>j}\left(\phi^{(1)}_{,ii}\phi^{(1)}_{,jj} - (\phi^{(1)}_{,ij})^2\right) \tag{7.4}$$

The 2LPT initial conditions are significantly more accurate than Zel'dovich for moderate-to-high redshift starts ($z_i \sim 50$).

Algorithm 7.1 (IC Generation)

1. Generate Gaussian random field $\delta(\mathbf{k})$ using input $P(k)$ and random seed.
2. Compute 1st-order potential: $\tilde{\phi}^{(1)}(\mathbf{k}) = -\tilde{\delta}(\mathbf{k})/k^2$.
3. (2LPT only) Compute 2nd-order source and potential $\phi^{(2)}$.
4. Assign displacements and velocities to particles on a uniform grid.
5. Migrate particles to their correct MPI domains.

8 PM Force Calculation

8.1 TSC Mass Assignment

The Triangular Shaped Cloud scheme assigns particle mass to the $3^3 = 27$ nearest grid points using a quadratic spline window:

$$W_{\mathrm{TSC}}(x) = \begin{cases} \frac{3}{4} - x^2 & |x| \leq \frac{1}{2} \\ \frac{1}{2}\left(\frac{3}{2} - |x|\right)^2 & \frac{1}{2} < |x| \leq \frac{3}{2} \\ 0 & |x| > \frac{3}{2} \end{cases} \tag{8.1}$$

The density field is:

$$\rho(\mathbf{x}_g) = \frac{m_p}{\Delta x^3} \sum_p W_{\mathrm{TSC}}(\mathbf{x}_g - \mathbf{x}_p) \tag{8.2}$$

The implementation in `tsc_omp2.c` uses OpenMP parallelization with thread-local density arrays to avoid race conditions.

8.2 FFT-Based Poisson Solver

Algorithm 8.1 (PM Force Pipeline)

1. **Density field** $\rho(\mathbf{x})$ computed via TSC.
2. **Forward FFT**: $\tilde{\rho}(\mathbf{k}) = \quad [\rho]$ using FFTW3-MPI (`fft.c`).
3. **Poisson solve**: $\tilde{\phi}(\mathbf{k}) = -\tilde{\rho}(\mathbf{k})/k^2$ (`p_solver.F`).
4. **Dark energy correction**: $\tilde{\phi} \to p_{\mathrm{corr}}(a) \cdot \tilde{\phi}$ for CPL models.
5. **Backward FFT**: $\phi(\mathbf{x}) = \quad ^{-1}[\tilde{\phi}]$.
6. **Force interpolation**: 4th-order FDA gives $\mathbf{F} = -\nabla\phi$ at particle positions (`fda4.c`).

8.3 4th-Order FDA

The force at grid point i is:

$$F_i = \frac{1}{12\Delta x}\left(-\phi_{i+2} + 8\phi_{i+1} - 8\phi_{i-1} + \phi_{i-2}\right) \tag{8.3}$$

This requires padding the potential mesh by 3 slices below and 4 slices above the local domain, handled by `paddingpotentialmesh()` in `VarPM.c`.

8.4 Variable PM (VarPM)

With `-DVarPM` enabled, each MPI rank can own a non-uniform number of z-slabs. The z-domain boundaries (`zmin`, `zmax`) adapt to balance particle counts. The TSC and FDA routines accept variable z-boundaries and convert between the local variable-width domain and the global uniform FFTW layout via `tsc_fftw_mesh()` in `VarPM.c`.

9 Tree Force Correction

9.1 Cell-Linked-List Construction

The tree correction uses a cell-linked-list rather than a full octree. In `ParticleCellLinkedList()` (`kjhtpm.mod1.gpu.omp.c`):

1. Divide the local domain into cells of width ~ 4 grid spacings.
2. Assign each particle to its cell using integer truncation of position.
3. Build a linked list through the `next` pointer of `treeparticletype`.

Particle positions near domain boundaries can produce out-of-range cell indices. The code clamps indices to valid range:

```
if(ix < 0) ix = 0;   if(ix >= mxm1) ix = mxm1;
if(iy < 0) iy = 0;   if(iy >= mym1) iy = mym1;
if(iz < 0) iz = 0;   if(iz >= mzm1) iz = mzm1;
```

9.2 Force Evaluation

For each particle, the tree correction computes the short-range gravitational force from nearby particles using a spline-softened kernel. The opening angle criterion θ determines whether to use a cell's center of mass or open it:

$$\frac{\ell}{r} < \theta \quad \Rightarrow \quad \text{use cell monopole} \tag{9.1}$$

where ℓ is the cell size and r is the particle–cell distance. The opening angle is adaptively adjusted during the simulation:

```
if(stepcount < 10)        theta = 0.3;
else if(stepcount < 20)   theta = 0.35;
else if(stepcount < 60)   theta = 0.4;
else                      theta = 0.45;
```

9.3 Type Conversion

Before tree correction, particles are expanded in-place from 32-byte `pmparticletype` to 40-byte `treeparticletype` (`pm2treetype`), and shrunk back afterward (`tree2pmtype`).

Algorithm 9.1 (pm2treetype — In-Place Expansion)

1. `Realloc` the particle array to $N_p \times 40$ bytes.
2. Copy backwards (from last particle to first) using `memmove` to avoid overwriting:

```
for(i = nplong-1; i >= 0; i--) {
    memmove((char*)treeparticles + i*sizeof(
        treeparticletype),
            (char*)pmparticles + i*sizeof(pmparticletype
                ),
            sizeof(pmparticletype));
}
```

3. The `next` pointer (last 8 bytes) is left uninitialized and set during linked-list construction.

10 Time Integration

10.1 Leap-Frog Scheme

The code uses a kick-drift-kick leap-frog integrator where the scale factor a advances by a fixed step Δa:

$$v_{n+1/2} = v_n + \frac{\Delta a}{2}\dot{v}_n \quad \text{(kick)} \tag{10.1}$$

$$x_{n+1} = x_n + \Delta a\, v_{n+1/2} \quad \text{(drift)} \tag{10.2}$$

$$v_{n+1} = v_{n+1/2} + \frac{\Delta a}{2}\dot{v}_{n+1} \quad \text{(kick)} \tag{10.3}$$

The velocity factors `fact1` and `fact2` encode the cosmological dependence:

$$\ddot{a} = -\frac{4\pi}{3}\left(\frac{1}{a^2} + (1+3w)\frac{\Omega_\Lambda}{\Omega_m}\frac{f_{\mathrm{DE}}(a)}{a^3}\right) \tag{10.4}$$

computed by `appapsq()` in `FLRW.F`.

10.2 Half-Step Synchronization

The `halfstep` structure controls when positions and velocities are synchronized (both at the same time level) for output:

- `halfstep.first = 1, second = 0`: First half-step (velocity kick only).
- `halfstep.first = 1, second = 1`: Second half-step (complete the full step).

  This is triggered by output flags (`flagsyncpdata`) and the PreFoF group finder.

11 Cosmology: CPL Dark Energy

11.1 CPL Parameterization

The equation of state for dark energy is parameterized as:

$$w(a) = w_0 + w_a\,(1 - a/a_{\max}) \tag{11.1}$$

where $a_{\max}$ is the maximum scale factor. The standard ΛCDM model corresponds to $w_0 = -1$, $w_a = 0$.

11.2 Expansion History

The Hubble parameter $H(a)$ in a flat universe:

$$H^2(a) = H_0^2 \left[\Omega_m \left(\frac{a_{\max}}{a}\right)^3 + (1 - \Omega_m - \Omega_\Lambda) \left(\frac{a_{\max}}{a}\right)^2 + \Omega_\Lambda\, f_{\rm DE}(a) \right] \tag{11.2}$$

where

$$f_{\rm DE}(a) = \left(\frac{a}{a_{\max}}\right)^{-3(1+w_0+w_a)} \exp\left[-3w_a \left(1 - \frac{a}{a_{\max}}\right)\right] \tag{11.3}$$

is the dark energy scaling function, implemented as `scaleDE()` in `FLRW.F`.

11.3 Growth Factor

The matter growth factor $D_+(a)$ satisfies:

$$\ddot{D}_+ + 2H\dot{D}_+ = \frac{3}{2}\Omega_m(a)\,H^2\,C_w(a)\,D_+ \tag{11.4}$$

where $C_w(a) = 1 + (1+w)\,\Omega_{\rm DE}(a)/\Omega_m(a)$ encodes the dark energy clustering. This is solved numerically via the RK4 integrator in `growthcpl()` with the `odeint` driver from Numerical Recipes.

11.4 Poisson Correction

For dark energy models with clustering ($c_s = 0$), the Poisson equation is modified:

$$\nabla^2\phi = 4\pi G\,\bar{\rho}_m\,\delta_m\,p_{\rm corr}(a) \tag{11.5}$$

where

$$p_{\rm corr}(a) = 1 + \frac{\Omega_{\rm DE}(a)\, D_{\rm DE}(a)}{\Omega_m(a)\, D_m(a)} \tag{11.6}$$

This correction is computed at each time step by `PoissonCor()` in `FLRW.F` and applied in `pmmain.c`.

For smooth dark energy ($c_s = 1$), $p_{\rm corr} = 1$ (no clustering correction).

12 Domain Decomposition and Migration

12.1 Slab Decomposition

The simulation box is divided into N_{rank} slabs along the z-axis. Each MPI rank owns a contiguous range of z-coordinates $[z_{\min}, z_{\max})$. This aligns with the FFTW3-MPI data layout, which distributes the first (z) dimension across ranks.

12.2 Particle Migration

After positions are updated, particles that have moved outside their domain are exchanged via `migrate()` in `pmigrate.kisti.omp.c`:

Algorithm 12.1 (Particle Migration)

1. Scan all local particles; classify as staying, moving up, or moving down.
2. Send "up" particles to rank+1, receive from rank−1 (periodic).
3. Send "down" particles to rank−1, receive from rank+1 (periodic).
4. Compact local array and append received particles.
5. If multi-slab migration is needed, repeat for larger shifts.

12.3 Load Balancing

The domain rebalancer in `mydomaindecomposition.omp.c` monitors the particle count ratio:

$$R = \frac{N_{\max} - N_{\min}}{N_{\text{avg}}} \times 100\% \tag{12.1}$$

If $R > 1\%$ (the `MAXINHOMO` threshold), z-slab boundaries are adjusted so each rank holds approximately equal numbers of particles.

IV

Reference

13 Troubleshooting

13.1 Common Issues

13.1.1 Segfault in Tree Correction

Symptom: Crash in `treecorrection()` or `ParticleCellLinkedList()`.

Possible causes:

1. **Strict aliasing violation in** `pm2treetype`: At `-O2`, the compiler may reorder reads/writes of `pmparticles[i]` and `treeparticles[i]` because they are different types. The fix is to use `memmove` through `char*` pointers.
2. **Out-of-bounds cell index**: Particles near domain boundaries can produce negative or overly large cell indices. Ensure clamping is applied.
3. **Insufficient memory**: If `NMEG` is too small, the `Realloc` in `pm2treetype` fails.

13.1.2 Malloc Error (Extremely Large Allocation)

Symptom: "Error allocating memory N bytes in Malloc()" where N is petabytes.

Cause: OpenMP clamping bug. Check that the y and z index limits use `mym1`/`mzm1` (not `mxm1`) in `ParticleCellLinkedList()`.

13.1.3 Power Spectrum File Not Found

Ensure the `Ascii Powerfile` path in `params.dat` is relative to the run directory, not the source directory.

13.2 Performance Tuning

- **NMEG**: Set large enough to hold all particles plus FFT workspace. Rule of thumb: NMEG $\geq 40 \times N_p^{1/3}$ MB per rank for N_p total particles.
- **MPI vs OpenMP balance**: Use fewer MPI ranks with more threads per rank to reduce communication overhead. Typically 4–8 threads per rank.
- **Opening angle θ**: Larger values are faster but less accurate. The adaptive scheme (0.3–0.45) is a good default.

14 Example: CPL Dark Energy Simulation

This chapter walks through a complete simulation with CPL parameters $\Omega_m = 0.31$, $w_0 = -1.2$, $w_a = -2.4$.

14.1 Parameter File

params.dat

```
#Start of the Ascii Header of the GOTPM Simulation
define INITIAL CONDITION    = 2
define OmegaMatter0         = 0.31
define OmegaBaryon0         = 0.044
define OmegaLambda0         = 0.69
define Hubble               = 0.72
define nPS                  = 0.96
define Boxsize(Mpc/h)       = 1024
define w0 of DE (CPL)       = -1.2
define wa of DE (CPL)       = -2.4
define Sound Speed of DE    = 0
define Amax                 = 48
define Anow                 = 1
define Astep                = 0.025
define Nx                   = 1024
define Ny                   = 1024
define Nz                   = 1024
define Nspace               = 1
define Theta                = 0.3
define Nstep                = 1881
define Stepcount            = 1
define Stepnum              = 1
define Iseed                = -56
define Powerflag            = 2
define Ascii Powerfile      = camb.z=47_0.31_-1.2_-2.4.ascii
define rvfilename           = INITIAL
define rvprefix             = INITIAL
#End of Ascii Header
```

14.2 Compilation

Rules.make Settings

```
CC = mpiicx
FC = mpiifx
OPT = -DINTEL -O2 -qopenmp
CDFLAGS = -DWGROUPSIZE=80 -DNMEG=8000L -DINCLUDE_TREE_FORCE \
          -D_LARGE_FILES -DSAVESLICE -DPMSEEDFORCE
COMFLAGS = -DINDEX -DVarPM -DXYZDBL -DMPI_SLOW
FFTWDIR = /home/kjhan/local/
```

14.3 Execution

With 16 MPI ranks and 4 OpenMP threads per rank (64 cores total):

```
export OMP_NUM_THREADS=4
srun --ntasks=16 --cpus-per-task=4 ./namu.exe params.dat
```

14.4 Expected Output

The simulation starts at $z = 47$ ($a = 1$) and evolves to $z = 0$ ($a = 48$) in 1881 steps. Key output messages:

- `Poisson Correction factor X at anow = Y` — Dark energy correction at each step.
- `CPU(F-FFTW3)` and `CPU(B-FFTW3)` — Forward/backward FFT timing.
- `Treecorrectingtime CPU= X` — Tree force correction timing.
- `Step N CPU= X` — Total time per step.

V

Post-Processing

15 FoF Halo Analysis

At multiple time steps during the simulation run, the code dumps FoF halo data (particle positions and velocities). The Friends-of-Friends algorithm identifies virialized halos using a linking length of 0.2 times the mean particle separation. The minimum halo mass is typically set to $m_h^c = 30 \times m_p$, where m_p is the particle mass.

15.1 Most Bound Particle (MBP)

Definition 15.1 (Most Bound Particle)

The MBP is the particle whose total energy is the most negative in a halo at the time the halo first forms. Once assigned, the MBP persists as long as it remains among the member particles of some halo. It serves as a proxy for the galaxy at the center of the halo.
The total energy of a particle is:

$$E = \frac{1}{2} m \left(\frac{d\mathbf{r}}{dt} \right)^2 + \Phi(\mathbf{r}) \tag{15.1}$$

where the velocity includes both peculiar velocity and Hubble flow. The gravitational potential is:

$$\Phi(\mathbf{r}) = m \sum_i \frac{G m_i}{\sqrt{(\mathbf{r}_i - \mathbf{r})^2 + \varepsilon^2}} \tag{15.2}$$

where ε is the force resolution and i runs over all member particles of the halo.

At each time step, MBPs identified in the previous step are checked against the current FoF catalog:

- If a halo contains no previously assigned MBP, a new MBP is selected by computing the total energy of all member particles.
- If one or more MBPs are already present, no new MBP is created.

15.2 Attached Quantities

The MBP is a particle with its own index, position, and peculiar velocity. Since the MBP represents a galaxy within its halo, additional halo properties are attached:

- Mean halo velocity $\mathbf{v}_h$
- Velocity dispersion σ_v
- Virial radius $r_{\rm vir}$
- Halo mass (FoF mass)

15.3 Center of Mass

The center of mass (CoM) of a halo is the mean position of its member particles. The group velocity is the average peculiar velocity.

15.4 Halo Shape

The shape of a halo is measured via the eigenvalues and eigenvectors of the shape tensor:

$$T_{ij} = \sum_k r_{k,i} r_{k,j} \tag{15.3}$$

where $\mathbf{r}_k$ is the displacement of particle k from the halo CoM. The eigenvalue ratios give the axis ratios of the halo.

16 MBP Merger Tree

The MBP merger tree tracks the hierarchical assembly of halos through cosmic time. Each MBP carries a unique Global Identification number (GID) and a history of merger events.

16.1 Basic Concepts

16.1.1 GID and Major GID

Definition 16.1 (GID)

The GID (Global Identification) is a unique number stored as `size_t` in C format. It is assigned when a halo first forms and its MBP is created.

Definition 16.2 (Major GID)

The Major GID is the GID of the most massive (central) MBP in a halo at each merger step. It may change with time as the central MBP changes.

16.1.2 Merger Events

The basic concept is to record each MBP's merger history in a single line. Merger events are marked by a change in the Major GID:

- When a halo first forms, it has one MBP whose Major GID equals its own GID.
- When halos merge, the MBP from the more massive progenitor becomes the *central* (major) MBP; the other becomes a *satellite* MBP.
- The satellite MBP's Major GID changes to the central MBP's GID.

16.1.3 Central and Satellite MBP

When two halos of different masses merge:

- The MBP from the more massive halo is the *central* MBP.
- The MBP from the less massive halo is the *satellite* MBP.
- The satellite's mass, shape, and radius are frozen at their values just before merging.
- The satellite's σ_v is set to the merger halo's σ_v.
- The satellite's position and velocity continue to be tracked via particle movement.

16.1.4 MBP Mass

The mass of an MBP equals its hosting halo mass when it is the sole or central MBP. For satellite MBPs, the mass is frozen at the hosting halo mass measured just before the merger event.

16.2 Data Structures

16.2.1 Original Merger Tree Format

Definition 16.3 (SavedHaloType)

```
typedef struct SavedHaloType {
    INDXTYPE mbp;                          /* MBP particle index
        */
    IDTYPE nowhid, majorglobalid;          /* halo ID, major GID
        */
    POSTYPE x, y, z;                       /* MBP position (h^-1
        Mpc, comoving) */
    POSTYPE hx, hy, hz;                    /* Halo CoM position
        */
    float mass, fofhmass;                  /* MBP/halo mass, FoF
        halo mass (M_sun/h) */
    float vx, vy, vz;                     /* MBP peculiar
        velocity (km/s) */
    float hvx, hvy, hvz;                  /* Halo peculiar
        velocity (km/s) */
    short int statusflag, linkflag;        /* Internal flags */
} SavedHaloType;
```

`POSTYPE` is `double` when compiled with `-DXYZDBL`, otherwise `float`. `INDXTYPE` is typically `long`.

Important

The field `majorglobalid` in `SavedHaloType` stores the Major GID, not a local halo index.

16.2.2 GID-Sorted Merger Tree Format

The sorted merger tree groups all tree lines belonging to the same $z = 0$ halo together, enabling efficient analysis of merger histories.

Definition 16.4 (GID-Sorted Data Structures)

```
typedef struct MajorGid {
    IDTYPE gid, newgid, mgid;
    IDTYPE majorgid[NSTEPS];
} MajorGid;

typedef struct GID {
    IDTYPE gid, newgid;
} GID;

typedef struct SigV {
    float sigv[NSTEPS];
} SigV;

typedef struct GidMergeHistory {
    IDTYPE gid;
```

```
        SavedHaloType step[NSTEPS];
        SigV sigv;
    } GidMergeHistory;
```

Here `IDTYPE` is `long` and `NSTEPS` is the number of merger tree time steps.

The output file contains:

1. A header with simulation parameters (box size, h, n_s, Ω_m, Ω_b, Ω_Λ, σ_8^{-1}, Δa, number of merger steps, step numbers, and redshifts).
2. An array of `GidMergeHistory` structures containing the full merger history.

16.3 Building the Merger Tree

Algorithm 16.1 (Merger Tree Construction)

1. **Extract halo properties**: Run the FoF halo finder to produce halo catalogs.

   ```
   mpirun -np <N> ./mpimf2.exe <step_number>
   ```

   This produces files including `FoFHaloQuantities.%.5d.bin`.
2. **Link MBPs between steps**: Connect MBPs across adjacent time steps.

   ```
   mpirun -np <N> ./sim2link.mpi.mod2.exe <step_before> <
       step_after>
   ```

3. **Build merger chains**: Assemble the linked MBPs into complete merger trees.

   ```
   mpirun -np <N> ./mkmerger.exe
   ```

4. **Combine output**: Merge all partial files into a single merger tree file.

   ```
   makeone
   ```

16.4 Sorting by Major GID

The merger tree lines are sorted by their Major GID at the final redshift (typically $z = 0$), so that all tree lines belonging to the same $z = 0$ halo are grouped together.

Algorithm 16.2 (GID Sorting)

1. **Assign new GIDs**: Re-index the merger tree by major GID.

   ```
   mpirun -np <N> ./SortMajorGid.exe <redshift>
   ```

   Source files: `SortMajorGid.c`, `pqsort.SMG.c`, `SMG.h`.
2. **Sort and dump**: Produce the final sorted output file.

   ```
   mpirun -np <N> ./Sort_SortMajorGid.exe <input_header> <
   ```

```
        num_files>
```

Source files: `Sort_SortMajorGid.c`, `pqsort.SMG.c`, `SMG.h`.

The internal structure of the sorted merger tree depends on the final redshift. When building, check the values of `iredshift`, `NSTEPS`, and `MSTEPS` in the source code to match your simulation.

16.5 Reading the Merger Tree

Example 16.1 (Reading Sorted Merger Tree Data)

```c
/* Compile: icc -o checkSGMT checkSGMT.c -DINDEX -DXYZDBL -g */
#include <stdio.h>
#include <stdlib.h>
#include "merger.h"
#include "mkmerger.h"
#include "SMG.h"
#define NSTEPS 131

int main(int argc, char *argv[]) {
    GidMergeHistory *bp;
    IDTYPE np;
    float boxsize, hubble, npower, omep, omepb, omeplam, bias, astep;
    int msteps, *nsteps;
    float *reds;

    FILE *fp = fopen(argv[1], "r");
    fread(&boxsize, sizeof(float), 1, fp);
    fread(&hubble,  sizeof(float), 1, fp);
    fread(&npower,  sizeof(float), 1, fp);
    fread(&omep,    sizeof(float), 1, fp);
    fread(&omepb,   sizeof(float), 1, fp);
    fread(&omeplam, sizeof(float), 1, fp);
    fread(&bias,    sizeof(float), 1, fp);
    fread(&astep,   sizeof(float), 1, fp);
    fread(&msteps,  sizeof(float), 1, fp);
    nsteps = (int*)malloc(sizeof(int) * msteps);
    reds   = (float*)malloc(sizeof(float) * msteps);
    fread(nsteps, sizeof(int),   msteps, fp);
    fread(reds,   sizeof(float), msteps, fp);

    bp = (GidMergeHistory*)malloc(sizeof(GidMergeHistory) * 10000);
    while ((np = fread(bp, sizeof(GidMergeHistory), 10000, fp)) > 0) {
        printf("%ld\n", bp[0].gid);
    }
```

```
    free(bp);
    return 0;
}
```

The input file is `SortByMajorGidMergingTree.NNNNN.dat`.

16.6 Known Issues

16.6.1 Disappearance of MBP

In rare cases, an MBP may be ejected from its host halo (e.g., into the intergalactic medium). When this happens, the MBP line terminates and its variables are set to null for subsequent steps. A new MBP is then selected in the halo.

16.6.2 Fluctuating Small Halos

When a halo's mass is near the critical threshold, numerical fluctuations can cause it to drop below the threshold at the next step. This may generate multiple MBPs for a single physical halo; only one will evolve further while the others terminate.

16.6.3 Null Values

A null, zero, or negative value in the `mbp` field indicates that the MBP was not found at that epoch.

16.6.4 Mass Conservation

The total mass before and after a merger event may differ due to mass loss during the merger process. Analysis should account for this discrepancy.

17 PSB Subhalo Finder

The PSB (Phase-Space Based) subhalo finder is a standalone MPI post-processing tool located in `SRC(FoF_PSB_Free)/`. It identifies gravitationally bound substructures within FoF halos by combining density peak detection, contour-based region separation, and iterative energy unbinding. The executable `hfind.exe` reads the FoF output produced in Chapter 15 and writes a subhalo catalog for each time step.

> `hfind.exe` is compiled and run independently from the main simulation. It requires only the FoF halo catalog and member particle files as input.

17.1 Algorithm Overview

Algorithm 17.1 (PSB Subhalo Identification Pipeline)

The subhalo finder operates in six stages:

1. **Density estimation**: Compute the SPH density at each particle position using an adaptive grid and the W4 cubic spline kernel.
2. **Peak detection**: Identify local density maxima using a 26-neighbor criterion.
3. **Density contour separation**: For each pair of neighboring peaks, perform a binary search in density to find the saddle-point contour that separates them. Use depth-first-search (DFS) flood-fill to assign particles to each peak's region.
4. **Energy unbinding**: Remove kinetically unbound particles via iterative tidal-radius and total-energy checks.
5. **FoF substructure linking**: Apply tree-accelerated Friends-of-Friends linking to associate remaining shell particles with the nearest bound core.
6. **Output**: Write the subhalo catalog and background (unbound) particles.

17.2 Density Estimation

The density field is computed on a sub-grid using the SPH W4 cubic spline kernel.

Definition 17.1 (SPH W4 Cubic Spline Kernel)

The smoothing kernel used for density estimation is:

$$W(r,h) = \frac{1}{\pi h^3} \begin{cases} 1 - \frac{3}{2}q^2 + \frac{3}{4}q^3, & 0 \le q < 1 \\ \frac{1}{4}(2-q)^3, & 1 \le q < 2 \\ 0, & q \ge 2 \end{cases} \tag{17.1}$$

where $q = r/h$ and h is the smoothing length. The implementation is in `find_density.f`.

The density grid has a maximum size of `MAX_BOX_MEMORY` $= 421{,}875{,}000$ cells (750^3). The grid dimensions are adapted to the bounding box of each FoF halo's member particles. MPI-parallel density estimation exchanges boundary layers (`nbuff` $= 5$ cells) between adjacent ranks via `MPI_Sendrecv`.

Algorithm 17.2 (Density Field Construction)

1. Compute bounding box of all particles in the halo.
2. Determine sub-grid cell size h and grid dimensions (n_x, n_y, n_z).
3. Assign each particle to the nearest grid cell.
4. For each particle, deposit its contribution to neighboring cells using $W(r,h)$.
5. Exchange boundary layers between MPI ranks.
6. Interpolate the grid density back to particle positions.

17.3 Peak Detection

Algorithm 17.3 (Density Peak Detection)

1. For each particle i with density $\rho_i >$ `peakthreshold`, examine all 26 neighbors in the density grid (the $3^3 - 1$ surrounding cells).
2. If ρ_i is strictly greater than all 26 neighbors, mark particle i as a density peak and set the `WP_PEAK` flag.
3. Peaks with fewer than `MIN_HALO_SEED_NUM` $= 10$ particles in their immediate neighborhood are discarded.

The maximum number of peaks per halo is `MAX_PEAK_NUM` $= 100{,}000$.

Peak Threshold

The parameter `peakthreshold` $= 1.5$ (set in `sub_find_dimension.inc`) controls the minimum overdensity required for a local maximum to be considered a density peak. Lower values produce more candidate peaks but increase computation time.

17.4 Density Contour Separation

Once density peaks are identified, the algorithm must separate overlapping regions. For each pair of adjacent peaks, a binary search in density threshold locates the saddle-point contour.

Algorithm 17.4 (Contour Separation via Binary Search)

1. For each peak, find the nearest neighboring peak.
2. Set the initial density search range: $[\rho_{\text{low}}, \rho_{\text{high}}] = [\texttt{downthreshold}, \min(\rho_{\text{peak},1}, \rho_{\text{peak},2})]$.
3. Bisect: set $\rho_{\text{mid}} = (\rho_{\text{low}} + \rho_{\text{high}})/2$.
4. Perform a DFS flood-fill from each peak, visiting only cells with $\rho > \rho_{\text{mid}}$.
5. If the two flood-fills meet (regions are connected), raise $\rho_{\text{low}} = \rho_{\text{mid}}$; otherwise lower $\rho_{\text{high}} = \rho_{\text{mid}}$.
6. Repeat until $|\rho_{\text{high}} - \rho_{\text{low}}| < \texttt{exactthreshold} = 0.2$.

The converged ρ_{mid} defines the core density threshold for each peak.

Definition 17.2 (DFS Flood-Fill)

Starting from a density peak, the depth-first search visits neighboring cells on the packed 3D grid. A cell is visited if its density exceeds the current threshold ρ_{mid} and it has not been previously visited (checked via the `WP_VISITED` flag). The maximum recursion depth is $\texttt{ndepth} = 10{,}000{,}000$.

17.5 Energy Unbinding

Particles in each core and surrounding shell are tested for gravitational binding. Unbound particles are iteratively removed.

Algorithm 17.5 (Iterative Energy Unbinding)

1. Compute the tidal radius R_{tidal} for each core from its mass and the background density.
2. For each particle within R_{tidal}, compute:
   - Kinetic energy: $K = \frac{1}{2} m |\mathbf{v} - \mathbf{v}_{\text{cm}}|^2$
   - Gravitational potential: $\Phi = -G \sum_j m_j / \sqrt{r_{ij}^2 + \varepsilon^2}$
3. Remove particles with $E = K + \Phi > 0$ (unbound).
4. Recompute the center of mass and tidal radius.
5. Repeat for $\texttt{BOUND_ITER} = 4$ iterations.

Cores are processed in order of increasing mass (smallest first).

Important

The ordering of cores during unbinding matters. Processing smaller cores first prevents them from being absorbed into larger neighbors before their own binding state is evaluated.

The potential calculation uses the spline-softened force kernel initialized from `force_spline.mod3.c`. The number of integration sub-steps is $\texttt{NSTEP} = 5$, and the potential is averaged over $\texttt{REPEAT_POTENT_NUM} = 4$ evaluations.

17.6 FoF Substructure Linking

After unbinding, remaining shell particles are associated with cores via tree-accelerated Friends-of-Friends linking.

Algorithm 17.6 (Tree-Based FoF Linking)

1. Build a spatial tree over all core and shell particles.
2. For each particle, walk the tree to find neighbors within the linking length $b = \texttt{FOF_LINK} \times \bar{d}$, where $\bar{d}$ is the mean particle separation and $\texttt{FOF_LINK} = 0.2$.
3. If two particles are linked, merge their group labels.
4. Iterate until no new links are formed.
5. Assign each FoF group to the core it contains (or the nearest core if none).

The tree walk is implemented in `fof.mod8.F` using the tree structures from `Treewalk.hfind.mod2.c`.

17.7 MPI Parallelization

The code uses a master-worker pattern. Rank 0 reads the FoF catalog and distributes individual halos to worker ranks for subhalo analysis.

Algorithm 17.7 (Master-Worker Communication Protocol)

1. **Broadcast**: Rank 0 broadcasts cosmological parameters (box size, h, Ω_m, Ω_Λ, scale factor) to all ranks.
2. **Distribution**: For each halo in the catalog:
   (a) Rank 0 probes for a `READY` message from any worker.
   (b) Rank 0 sends the particle count (`NP_TAG`), positions (`R_TAG`), and indices (`INDX_TAG`) to the ready worker.
3. **Processing**: Each worker calls `subhalo_den()` on the received halo data.
4. **Collection**: The worker sends back the particle-to-halo mapping (`PT2H_TAG`) and the number of subhalos found (`MPEAK_TAG`).
5. **Output**: Rank 0 writes the results for each halo to disk (`WRITING` tag signals write phase).

The following MPI tags are used for inter-rank communication:

Tag Name	Value	Purpose
`READY`	1	Worker signals readiness
`WRITING`	3	Write-phase synchronization
`NP_TAG`	17	Number of particles in halo
`INDX_TAG`	24	Particle index array
`R_TAG`	32	Position data (double)
`PT2H_TAG`	55	Particle-to-halo mapping
`VR_TAG`	83	Velocity data (float)
`MPEAK_TAG`	96	Number of subhalos (peaks) found
`DEN_TAG`	100	Density data
`PEAK_DEN_TAG`	108	Peak density values

17.8 Data Structures

Definition 17.3 (basicparticletype — 40 bytes)

```
typedef long indxtype;
typedef struct basicparticletype {
    double x, y, z;        /* position (double precision) */
    float vx, vy, vz;      /* velocity (single precision) */
    indxtype indx;          /* particle index */
} basicparticletype;
```

Defined in `defs.h`. This is the primary particle type used throughout the subhalo finder. Positions are stored in double precision for accuracy; velocities in single precision to save memory.

Definition 17.4 (Halo)

```
typedef struct Halo {
    size_t np;              /* number of particles */
    double x, y, z;         /* center position */
    float vx, vy, vz;       /* center velocity */
} Halo;
```

Defined in `hfind.mod1.c`. Stores the properties of each FoF halo read from the catalog.

Definition 17.5 (Coretype)

```
typedef struct Coretype {
    int peak;               /* peak particle index */
    int nummem;             /* number of member particles */
    float coredensity;      /* core density threshold */
    float cx, cy, cz;       /* center of mass position */
    float cvx, cvy, cvz;    /* center of mass velocity */
    float Rtidal;           /* tidal radius */
    unsigned char flag;     /* status flags */
} Coretype;
```

Defined in `subhaloden.mod3.c`. Each density peak that survives the filtering stages becomes a `Coretype` entry.

Key arrays used during processing:

- `den[]` — SPH density at each particle position.
- `den2ptl[]` — mapping from density-sorted index to particle index.
- `ptl2halonum[]` — final particle-to-subhalo assignment (-9999 if unbound).
- `ppoint[]` — linked-list pointers for region traversal.

Working particle flags (bit flags in `subhaloden.mod3.c`):

Flag	Value	Meaning
`WP_RESET`	`0x00`	No flags set
`WP_PEAK`	`0x01`	Particle is a density peak

Flag	Value	Meaning
WP_VISITED	0x02	Visited during DFS flood-fill
WP_CORE	0x04	Assigned to a core region
WP_SHELL	0x08	Assigned to a shell region
WP_BOUND	0x10	Gravitationally bound to a core
WP_REMAINING	0x20	Unassigned after all stages

17.9 Parameters

17.9.1 Compile-Time Parameters (header.h)

Parameter	Default	Description
MAX_PEAK_NUM	100 000	Maximum density peaks per halo
MIN_HALO_SEED_NUM	10	Minimum particles in a halo seed
FOF_LINK	0.2	FoF linking length (mean separation units)
NSTEP	5	Potential integration sub-steps
BOUND_ITER	4	Energy unbinding iterations
REPEAT_POTENT_NUM	4	Potential averaging evaluations
TIDAL_STRIDE_NUM	1000	Stride for tidal radius sampling
MAX_BOX_MEMORY	421 875 000	Maximum density grid cells (750^3)
SNGL_CPU_PTL_NUM_MAX	10^8	Maximum particles per CPU
SERIAL_MAXIMUM	10 000	Threshold for serial processing

17.9.2 Density Grid Parameters (sub_find_dimension.inc)

Parameter	Default	Description
tthreshold	1.5	Density threshold for region growing
peakthreshold	1.5	Minimum density for peak candidacy
downthreshold	20.0	Lower bound for contour binary search
exactthreshold	0.2	Convergence criterion for binary search
ndepth	10 000 000	Maximum DFS recursion depth
ndimension	27	3D neighborhood size (3^3)

17.9.3 Memory Configuration (Rules.make)

Parameter	Default	Description
NMEG	1700	Memory pool per MPI rank (MB)
-D_LARGE_FILES	—	Enable large file support

17.10 Input and Output Formats

17.10.1 Input Files

The subhalo finder reads two binary files produced by the FoF halo finder (Chapter 15):

- `FoF_halo_cat.NNNNN` — FoF halo catalog header containing simulation parameters (box size, h, Ω_m, Ω_b, Ω_Λ, w_0, w_a, grid size, scale factor) and the number of halos.
- `FoF_member_particle.NNNNN` — Binary dump of member particles for each FoF halo (positions, velocities, and indices as `basicparticletype`).

Positions in the input files are in grid units (integer grid index). The code converts to physical units (h^{-1} Mpc, comoving) using the box size and grid dimensions read from the catalog header. Velocities are stored as simulation-internal values and converted to km s^{-1} via the scale-factor-dependent conversion factors `r1kineticfact` and `r2kineticfact`.

17.10.2 Output Files

- `HFIND.DAT.NNNNN` — Binary subhalo catalog. For each FoF halo, the file contains:
  1. `int mpeak`: number of subhalos found.
  2. For each subhalo $i = 0, \ldots, \texttt{mpeak} - 1$:
     - `int wnp`: number of member particles.
     - `double x,y,z` $\times$ `wnp`: positions (physical units).
     - `double vx,vy,vz` $\times$ `wnp`: velocities (km s^{-1}).
     - `indxtype indx` $\times$ `wnp`: particle indices.
- `background_ptl.NNNNN` — Particles not bound to any subhalo (same binary format as above, without subhalo grouping).

17.11 Building and Running

Compilation

```
cd SRC(FoF_PSB_Free)/
# Edit Rules.make: set NMEG, compiler paths
vi Rules.make
make all
```

Execution

```
mpirun -np <N> ./hfind.exe <step_number>
```

Prerequisites

Before running `hfind.exe`, ensure that the FoF halo output files (`FoF_halo_cat.NNNNN` and `FoF_member_particle.NNNNN`) exist in the working directory. These are produced by the FoF analysis described in Chapter 15.

The default `Rules.make` uses `mpiifort` and `mpiicc` (Intel classic compilers). For Intel oneAPI 2025+, change to `mpiifx` and `mpiicx`. The `-mcmodel=medium` flag is required for large memory allocations.

17.12 Source File Reference

File	Description
`hfind.mod1.c`	Main driver: MPI setup, I/O, master-worker loop
`subhaloden.mod3.c`	Core subhalo algorithm: density peaks, cores, unbinding
`find_density.f`	SPH density estimation (Fortran, W4 kernel)
`hfind.sub.F`	Halo seed finding, binary search separation
`hfind.sub.mod3.F`	Extended halo seed routines
`fof.mod8.F`	FoF substructure linking (Fortran)
`find_near.c`	Near-neighbor search
`Treewalk.near.c`	Tree walk for neighbor finding
`Treewalk.hfind.mod2.c`	Tree walk for subhalo FoF linking
`force_spline.mod3.c`	Spline-softened force kernel initialization
`Memory2.c`	Custom memory pool manager
`header.h`	Compile-time parameters and constants
`defs.h`	Type definitions (`basicparticletype`, macros)
`tree.h`	Tree node and particle structures
`sub_find_dimension.inc`	Density grid parameters (Fortran include)